

TrainVerify：由 Lean 内核检查的编译器生成分布式训练图等价性

中文技术报告初稿 v0.2

TrainVerify 项目

2026 年 8 月 11 日

摘要

分布式训练编译器把单设备训练程序改写成多设备局部程序，并插入张量分片、复制、通信、布局变换与调度。程序能够运行、张量形状一致或少量随机测试通过，都不能说明改写前后计算的是同一个值：两个局部张量可能拥有完全相同的形状，却分别代表连续序列分片、首尾交错的 zigzag 分片、复制副本或不同专家持有的 token。TrainVerify 将这类正确性问题转换为 Lean 可检查的关系命题。系统从固定源码和真实双 GPU 运行中取得单设备参考图 SM 与并行执行图 PM，把图节点、张量值、设备所有权和 collective 翻译成形式语义，沿目标的完整上游依赖生成命题，再把局部算子关系组合为图级证明。

本文从分布式训练的基本对象开始，解释 llm-train 如何定义 YOCO 模型，nnScaler 如何 trace、规划、插入 adapter 和 collective 并生成 SM/PM，以及这条转换为什么容易产生 shape-correct but value-wrong 的错误。随后给出 TrainVerify 的图翻译、人工 fidelity audit、命题构造、proof-certificate composition、信任边界和发布机制。YOCO-MoE-A0.4B 案例在固定 revision 的真实双 GPU authority 上得到五个公开 full theorem；最终快照直接检查 463 个 Lean 模块，并以逐文件 registry 和 receipt 绑定图来源、证明源码与检查结果。该结果证明的是项目张量语义下五个固定 observable 的关系，不是 CUDA kernel 的逐位等价、完整训练收敛或任意网络的一键验证。本文最后把已经通用、设计上应通用但尚未完成、以及必须由模型或框架专家单独投入的工作明确分开。

1 引言

考虑长度为 8 的序列。普通两路切分可以让 rank 0 持有行 $[0, 1, 2, 3]$ ，rank 1 持有 $[4, 5, 6, 7]$ ；按 rank 顺序执行 AllGather 能恢复原序列。context parallelism 使用的 zigzag 布局却可能让两个 rank 分别持有 $[0, 1, 6, 7]$ 与 $[2, 3, 4, 5]$ 。两种情况下，每个局部张量的 shape 完全相同，但普通 rank-order concat 在后一种布局下得到

$$[0, 1, 6, 7, 2, 3, 4, 5],$$

而不是原序列。通信调用可以成功，输出 shape 也正确，错误却已经进入值语义。

这不是孤立的排列问题。现代训练框架会组合 tensor parallelism、expert parallelism、context parallelism 和 data parallelism。编译器需要决定每个张量由哪些 rank 持有、沿哪一维切分、何时复制、何时 reduce，以及在两种布局之间插入什么 adapter。MoE 还要处理 token routing、expert ownership 和 AllToAll；ring 或 sliding-window attention 还会让 query stream 与 cache K/V 采用不同布局。任何一层的 annotation、partition、input order、process-group scope 或重构规则出错，都可能得到“能运行但计算了另一个函数”的程序。

传统测试覆盖了其中一部分风险。shape checking 能发现维度不一致；CPU smoke 能发现 import、trace 和 code generation 失败；数值 differential testing 能在有限输入上比较输出。但这些方法无法穷尽所有 rank-local 值关系，也很难判断测试没有触发的布局分支。另一方面，直接对训练框架源码做完整形式化成本过高，而且真实并行计划依赖 compiler revision、cost profile、硬件和 runtime path。TrainVerify

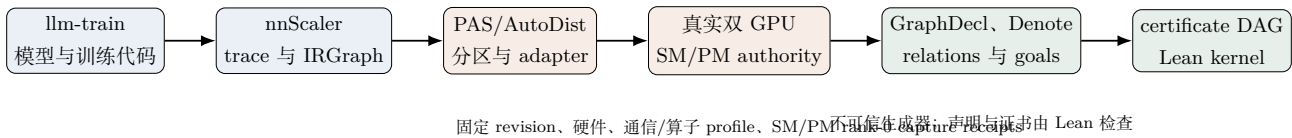


图 1: 从训练源码到内核检查证明的端到端链。

选择一条 translation-validation 路线：固定一次真实编译得到的 SM/PM 图，为该图建立值语义，并由 Lean 检查“PM 的局部值能否按声明的布局关系重构为 SM 值”。

本文作出以下贡献。

1. 给出一种面向分布式训练图的值级规格。它不仅记录 shape，还显式区分连续分片、zigzag、复制、expert ownership、hidden sharding 和 collective scope。
2. 给出 authority-backed graph-to-Lean pipeline。系统将真实 llm-train/nnScaler 编译产物、硬件与 profile provenance、图结构和目标 observable 转换为声明、输入条件和 proof obligations。
3. 给出从局部算子关系到完整目标上游依赖的证明组织方法。命题从真实外部输入开始，不允许把图计算产生的中间值作为未经约束的 premise。
4. 在 YOCO-MoE-A0.4B 的真实双 GPU 图上闭合五个公开 full theorem，并建立 theorem-header、axiom、non-vacuity、all-source build、registry 和 sealed receipt 门禁。
5. 系统分析当前方法的泛化边界，区分可复用语义库、尚未自动化的 relation inference 与 certificate synthesis，以及新算子、新布局和新训练框架不可避免的人工 fidelity audit。

图 1 给出全文主线。需要特别注意，Lean theorem、GPU authority 和 sealed publication 是三层不同证据。Lean 检查 proof term 是否证明给定声明；authority 说明声明里的图来自哪次真实编译；publication 说明被检查和被发布的是同一组字节。任何一层都不能替代另外两层。

2 分布式训练入门

2.1 一次训练迭代里有哪些对象

为了理解 SM 和 PM，不需要先掌握完整的深度学习数学。一次训练迭代至少包含四类对象。

第一类是**外部输入**，例如 token、label、packed sequence metadata 和 attention mask。第二类是**参数**，例如 embedding matrix、attention projection 和 MoE expert weights。第三类是**中间张量**，它们由 forward operators 依次产生。第四类是**梯度与更新状态**，由 loss 反向传播得到。编译器 trace 的图不一定只包含模型论文里的“层”；它还可能包含 reshape、stack、autograd、gradient accumulation、通信和为布局转换插入的 adapter。

YOCO-MoE-A0.4B 中与本文最相关的 forward 路径包括 embedding、24 个 decoder layers、attention、MoE router、expert grouped GEMM、residual、RMSNorm 和 cross-entropy head。context parallelism 会改变序列行的设备所有权，expert parallelism 会改变 token 与 expert weight 的设备所有权。两者叠加后，某个 attention layer 可以同时消费 zigzag 的 query stream 和普通连续分片的 cache K/V。

2.2 rank、shard 与 collective

rank 是一次分布式执行中的工作进程，通常绑定一张 GPU。一个全局 tensor 可以被复制到所有 rank，也可以沿某个维度分片。**collective** 是多个 rank 共同参与的通信，例如：

- AllGather：收集各 rank 的 shard；

- AllReduce: 聚合并把结果发送给所有 rank;
- ReduceScatter: 聚合后再分片;
- AllToAll: 每个 rank 向每个 rank 发送不同分块。

collective 的名字本身不决定正确性。AllGather 必须知道 concat 维度和 rank 顺序; AllToAll 必须知道输入如何切块、输出如何排列; process group 还决定哪些 rank 参与。只要其中一个约定与 producer 的实际 ownership 不一致, 就可能产生 shape 正确的错误值。

2.3 常见并行方式

方式	核心理念	对正确性规格的影响
Data parallelism	不同 rank 处理不同样本, 参数或梯度同步	需要 batch partition 与 gradient reduction 关系
Tensor parallelism	沿 hidden、head 等维度切分算子	需要 shard、partial result 和 gather/reduce 关系
Pipeline parallelism	不同 rank 执行不同层	需要 stage boundary 与调度顺序
Context parallelism	沿 sequence 切分 attention	需要 sequence ownership 与 permutation 关系
Expert parallelism	不同 rank 持有不同 experts	需要 routing、token dispatch 与 AllToAll 关系

表 1: 并行方式改变的不只是 shape, 还包括值的设备所有权和重构方式。

3 SM 与 PM 图

3.1 SM 图包含什么

本文把单设备参考图称为 **SM graph**。SM/PM 是 TrainVerify 对两次独立 nnScaler 编译结果的命名, 不是 nnScaler 上游定义的原生对象。SM 也不是手写数学规格, 而是同一 Backbone 子模块在单设备计划下 trace、规划和 code generation 得到的参考执行图。抽象上, 一张图包含:

- 按执行顺序排列的节点;
- 每个节点的 operator、输入与输出 tensor ID、参数;
- tensor shape 和 producer-consumer edge;
- rank、replica group 和 process-group metadata;
- 外部输入与目标 observable;
- 与源码 revision、pickle 和 profile 绑定的 provenance。

SM 的作用是提供未经目标并行计划改写的参考计算。它不要求与 PM 节点一一对应。一个 SM 节点在 PM 中可以展开成多个 rank-local 节点, 也可以由局部算子和 collective 的组合实现。

3.2 PM 图新增什么

PM graph 是并行规划器生成的多 rank 执行图。与 SM 相比, 它新增或改变:

- 节点在哪些 rank 上执行;

- tensor 是复制、连续分片、zigzag、expert-local 还是 partial reduction;
- 参数和 activation 的 placement;
- 为布局不匹配插入的 Chunk、AllGather、AllReduce、AllToAll 等 adapter;
- process group、replica-buddy 和 rank-local topology;
- runtime 需要的通信与算子 profile。

图 2 用一个简化 linear 例子展示二者差异。SM 直接计算 $y = xW$; PM 把 W 沿输出维切成两块, 每个 rank 计算一个局部 y_r , 随后根据下一节点的需要保留 shard 或执行 gather。正确性要求不是两个图长得一样, 而是局部输出满足一个可重构关系。

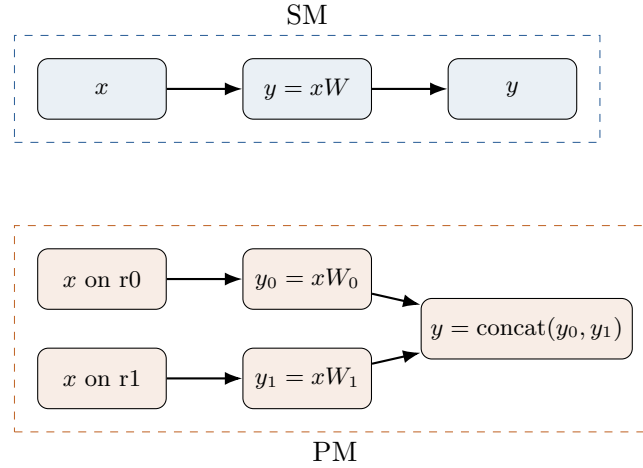


图 2: SM 与两 rank PM 的简化示例。节点不一一相等, 目标是局部值可重构。

4 llm-train 与 nnScaler 如何生成 SM/PM

4.1 llm-train: 被编译的程序

llm-train 定义 YOCO 模型结构、forward 调用、custom operators、训练输入以及 context/expert parallelism 所需的 API。真实入口不是一个独立的 `Trainer.compile()` 方法。`nnscaler_train.py--run_modecompile` 构造 `TrainerArgs` 后调用 `Trainer.run`; `Trainer._setup` 在 `compile-only` 分支以 `load_module=False` 调用 `parallelize_model`, 再进入 `ModuleParallelizeConfigAdapter.parallelize`、`nnscaler.parallel.parallelize` 和 `_generate`。

被并行化的对象是 `BackBone` 子模块, 不是完整 `WrapperModel`。llm-train 把 `BackBone` 注册为唯一 `ModuleParallelizeConfig`; nnScaler 在构造 wrapper 时截获该子模块。`BackBone.forward` 输出 token loss、routing maps 和 expert probabilities, 外层 `WrapperModel.forward` 对标量总 loss 的汇总不在捕获的 `ModuleCodeGen` 内。最终 SM/PM 因而不包含 optimizer update、dataloader 或完整训练循环。

llm-train 还通过 wrapper 或 annotation 告诉 nnScaler 某些 custom op 的输入输出结构。这里必须区分三件事: 模型代码规定数学数据流; annotation 描述编译器可以怎样理解和切分 custom op; runtime kernel selector 决定当前硬件实际尝试哪些实现。三者属于不同责任层。

例如, `wrap_maybe_shuffle(h)` 是否执行真实 permutation, 取决于 context-parallel process group 和 world size。K/V projection 在该调用之前还是之后, 决定 cache 的 ownership。仅从变量名、所在 layer 或最终 shape 无法可靠推断这一点, 必须沿源码和图中的 producer edge 检查。

4.2 nnScaler: 从程序到 IRGraph

llm-train 的 dummy-sample generator 构造 tokens、targets、positions、packed cu_seqlens、loss masks、indices 和 length metadata, 再由 backbone_forward_args_gen_fn 包装成 samples 参数。nnScaler 把子模块和这些 dummy tensors 移至 CPU, 运行 concrete_trace, 把 custom op 注册为 leaf, 随后将 FX graph 转换成 IRGraph。

forward graph 来自 trace; backward 不是一次真实 PyTorch 训练反传的记录。由于 BackBone 作为非 end-to-end 子模块编译, nnScaler 随后显式调用 IRGraph.backward(None), 逆序遍历 forward nodes 并创建 mirror backward ops。捕获的 ModuleCodeGen 因而包含自动生成的 backward 节点。

当前 Lean release 的边界更窄: 最终 GeneratedYOCOmoE.lean 与五个公开 Goal 只声明并证明 forward、adapter 和 collective 组成的目标 ancestry, 没有把这些 backward 节点翻译进公开 GraphDecl。本文只能声称 authority 捕获了 backward 结构, 不能声称当前五个 theorem 验证了 backward 正确性。

随后, PAS/AutoDist 读取用户约束、硬件规模、通信 profile 和算子 computation profile, 运行带内存界和通信/计算代价的受约束动态规划搜索, 并采用该搜索返回的候选计划。这里不能仅凭 pas_autodist 名称声称获得数学意义上的全局最优并行方案。

计划为计算节点分配 device 并执行 partition 或 replicate 后, IRAdapterGener 展开 multiref、生成 activation adapters 与 weight reducers。ExecutionPlan 按 device 建立执行序列; DiffFusion 与 Grouping 进一步整理图; ModuleCodeGen 再为每个 runtime rank 生成 gencode<rank>.py。nnScaler 原生 _gencode 不生成 mgener.pkl。TrainVerify 使用最小受控补丁, 在 ModuleCodeGen 构造后、逐 rank 代码生成前, 由 local rank 0 把该对象 dill 序列化为 sm_mgener.pkl 或 pm_mgener.pkl, 并把补丁和 pickle digest 写入 receipt。

SM 与 PM 来自同一 BackBone 源码、输入配置和被认证的 profile, 但它们不是先生成 SM 再给它追加通信节点。production generator 分别执行两次完整 compile: SM 使用 plan_ngpus=1, runtime_ngpus=1, zero_group_size=1; PM 使用 plan_ngpus=2, runtime_ngpus=2, zero_group_size=2。两次都重新经历 trace、IRGraph backward、PAS、adapter、ExecutionPlan 和 ModuleCodeGen。只检查 PM 无法知道某个 collective 到底是在实现参考值还是在制造新的值, 因此 authority 和后续验证必须同时保留两侧。

4.3 profile 为什么也是语义链的一部分

AutoDist 的选择依赖通信和算子成本。缺失 profile 时使用另一设备的 fallback, 或者让正式编译在运行中继续测量并修改 cache, 都可能得到另一张 PM 图。TrainVerify 因而把 communication profile、computation profile、GPU inventory 和 planner 配置固定到 authority provenance。profile 不直接证明数学正确性, 但它决定被证明的是哪张图。

4.4 从 nnScaler IR 到 verifier DFG

TrainVerify 的 Verdict backend 需要把 nnScaler 产物规范化为适合验证的数据流图。这一步不是机械 pickle 序列化。实现需要展开 segment 与 adapter、按 rank 展开节点、规范化 operator、补齐 backward 所需的 forward inputs、处理 constants、强制 single-static-assignment、注入 gradient accumulation, 并恢复 shape/value map 与 logical replica identity。

这也是一个高风险翻译边界。历史上实际发现过 reshape target shape 丢失、broadcast shape 用“较长 shape”近似、MoE hidden dimension 从错误参数取得、top-k expert count 解析错误、仅凭 shape 推断 ordinary gather、以及 evaluator selector 只看局部 slice 等问题。它们说明“图能导出”不是翻译正确性的证据。

5 为什么转换容易出错

5.1 shape 不携带 ownership

图 3 展示同一个全局序列的两种两路布局。普通 layout 的重构函数是 rank-order concat; zigzag layout 的重构函数需要把首尾 chunks 重新交错。两种局部 tensor shape 相同, 但 relation 不同。

SM	0	1	2	3	4	5	6	7
ordinary	0	1	2	3	4	5	6	7
zigzag	0	1	6	7	2	3	4	5

图 3: 相同局部 shape 可以对应不同所有权。颜色表示两个 rank。

5.2 mixed-layout attention

YOCO 后半段 decoder 是本项目最容易被错误概括的路径。shuffle 后的 Q 与 decoder residual stream 使用 zigzag relation; K/V projection 却在 shuffle 之前完成, 因此 cache K/V 仍是 ordinary contiguous shards。attention theorem 必须同时消费 zigzag Q、ordinary K、ordinary V、packed sequence metadata 和真实 reshape semantics。把“同一 layer 里的 tensor”统一标成 zigzag, 或者把两个 rank 的 K/V 当复制值, 都与实际 producer 顺序冲突。

5.3 MoE ownership

MoE router 根据 token 计算 top-k expert indices 与 scores; token 随后被发送到持有对应 expert weights 的 rank。正确性关系需要覆盖 routing map、dispatch mask、remote expert input、grouped GEMM、activation、AllToAll 和 output merge。相似的 operator 名或 TID 不足以证明两个 rank 值相等, replica fact 必须来自共同 source、明确 collective、replica-buddy metadata 或可检查 graph certificate。

5.4 跨层责任边界

一次失败可能来自: llm-train 的模型调用或 annotation, nnScaler 的 IR 或 adapter, runtime selector, Verdict normalization, Lean operator denotation, relation choice, statement generation, 或者 proof composition。TrainVerify 的失败结果必须保留这些层次, 而不是把所有问题统称为“模型 bug”或“证明失败”。

6 问题定义

6.1 图与执行语义

抽象上, tensor store 是从 tensor ID 到 tensor 值的映射:

$$\text{Store} = \text{TensorId} \rightarrow \text{Tensor}.$$

图 $G = [n_1, \dots, n_k]$ 按节点顺序解释。每个节点读取输入 tensor, 调用该 operator 的 denotation, 再把输出写回 store:

$$S_{i+1} = \text{evalNode}(n_i, S_i).$$

目标 t 的图语义记作

$$\text{denoteGraph}(G, S_0, t) = S_k(t).$$

对于依赖另一 rank 值、replica group、buddy rank 或 permutation 的节点，普通 rank-local evaluator 不足。faithful distributed evaluator 在执行时显式读取跨 rank 结构。

6.2 关系式正确性

TrainVerify 不要求 $G_{SM} = G_{PM}$ 。对目标 observable o ，证明形式是：若初始 SM store 与各 PM rank store 满足输入条件 C ，则执行两侧完整上游依赖后，结果满足 relation R_o ：

$$\forall S_{SM}, S_{PM}^0, S_{PM}^1, \quad C(S_{SM}, S_{PM}^0, S_{PM}^1) \Rightarrow R_o(\text{eval}(G_{SM}, o), \text{eval}(G_{PM}, o, 0), \text{eval}(G_{PM}, o, 1)).$$

R_o 可以表示 concat、zigzag reconstruction、复制相等、hidden-shard reconstruction 或更复杂的 collective 结果。

6.3 威胁模型与非目标

系统试图发现或拒绝：operator denotation 缺失、relation rule 缺失、ownership 歧义、错误 collective、computed premise 泄漏、statement drift、certificate bug、unexpected axiom、stale artifact 和 provenance mismatch。

系统当前不证明：

- CUDA kernel 的逐位浮点等价；
- compiler lowering 到 PTX/SASS 的 refinement；
- 完整训练收敛、最终模型质量或所有 optimizer state 一致；
- 任意 dynamic control flow 或任意 world size；
- translator 本身已被形式化验证。

7 TrainVerify 总体设计

7.1 Authority capture

production authority 必须来自真实双 GPU、双 rank NCCL compile path。仅设置 `plan_ngpus=2`、运行 CPU/Gloo smoke、手写 PM graph 或使用单 GPU 都不能替代。真实路径会触发 process group、shuffle、NCCL、GPU profile 和硬件 selector；这些分支可能决定图的 topology。

authority 固定 llm-train、nnScaler 和 TrainVerify revision，记录 GPU、driver、CUDA、NCCL、通信 profile、算子 profile、SM/PM 各自的 rank-0 capture receipt、pickle hash 与节点数。原始 pickle 是可执行输入，emitter 只接受本地产生、权限受控并由 out-of-band hardware digest 确认的工件。

7.2 不可信的声明与证书生成

Python emitter 生成 GraphDecl、shape environment、init goals、input value classes、intermediate obligations、observable goals、manifest 和 path-to-digest ledger。生成器不在 proof core 里；它可以产生任意声明，但不能让 Lean 接受一个类型不正确的 proof term。另一方面，Lean 也不会自动判断声明是否忠实翻译了 authority，因此 graph/semantic fidelity 需要独立审计和 mutation tests。

7.3 语义库与关系库

项目为 embedding、linear、RMSNorm、activation、reshape/view、stack、loss、top-k、MoE grouped GEMM、shuffle/unshuffle、attention 和 collective 等建立 denotation，并证明局部 relation-preservation theorems。规则的一般形式是：

$$R_1(x, x_0, x_1) \wedge \cdots \wedge R_m(z, z_0, z_1) \wedge P \Rightarrow R_{out}(f(x, \dots, z), f_0(\dots), f_1(\dots)),$$

其中 P 是 shape、axis、process group 或 metadata side conditions。

7.4 命题与证书

设计目标是：目标生成器从 observable 同时向后遍历 SM 和 PM producer edges，直到只剩真实外部输入；全图 relation inference 为每条相关 tensor edge 选择 typed relation；rule selection 生成局部 obligation；certificate composer 再按 producer order 组成 proof DAG。当前 YOCO 实现只完成了其中一部分：graph declaration、目标 slice、backward closure、provenance 和若干有限 graph certificates 可以自动生成；relation 选择与长链 composition 仍混合使用通用 theorem、模板化生成模块和 YOCO-specific 手写 proof modules。项目尚未实现对任意受支持图的全图 relation inference 与完整 certificate synthesis。失败时返回结构化诊断仍是设计目标，不是当前所有路径已有的能力。

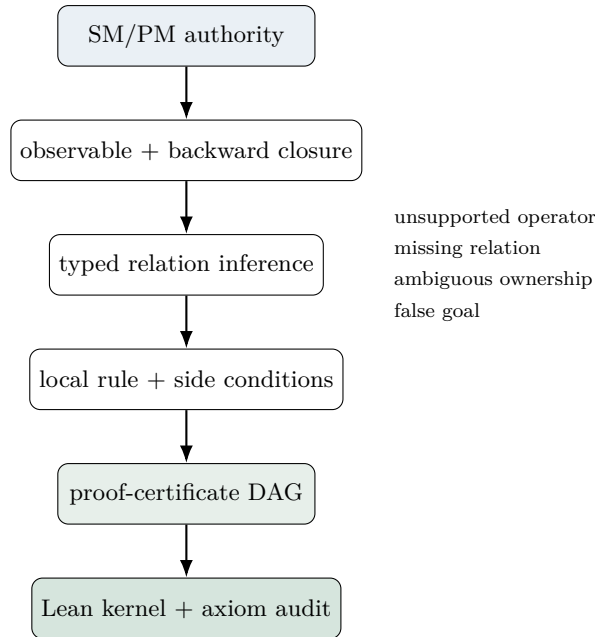


图 4: 从 authority graph 到 kernel-checked certificate 的逻辑阶段。

8 翻译忠实性与人工审查

8.1 为什么必须有人检查翻译

Lean 证明的是形式化声明。如果 Python operator 实现为 f ，Lean 却把它翻译成另一个函数 g ，那么 g 上的完美证明不能说明 f 。因此每个新 operator、layout 和 graph adapter 都需要 paper-to-source map。至少要检查：上游源码位置、input/output contract、shape 分支、值公式、world size、process-group scope、Lean declaration、最小可执行例子和 mutation sensitivity。

上游对象	Lean 对象	自动检查	人工检查
模型/custom op	operator denotation	signature、shape、fixture	值语义与 Python 源码一致
rank-local tensor	relation instance	producer、rank、shape	ownership、排列和复制来源
collective	transition theorem	group、axis、world size	与 runtime semantics 一致
目标 slice	goal graph	backward closure	没有漏 graph-computed input
输入条件	theorem premise	type 与 witness	条件没有偷渡结论
proof source	registry entry	exact path 与 digest	public theorem owner 正确

8.2 小张量对照与 mutation

可执行小张量对照能检测常见翻译错误。例如用行号 tensor 检查 shuffle/gather permutation, 用非对称 shape 检查 broadcast 和 reshape, 用不同 rank 值检查错误 replica assumption。mutation suite 应改变 operator、参数、input order、collective axis、ownership relation 和 certificate, 确认系统在预期阶段拒绝。它们提高 fidelity 信心, 但仍不等于 translator 已被形式化证明。

8.3 历史上暴露的 verifier-side 错误

YOCO 工作实际暴露了多项 verifier-side 问题: MoE grouped GEMM 的 hidden dimension 取错; broadcast 输出 shape 近似错误; top-k experts 参数解析错误; reshape target shape 丢失; 仅凭 shape 推普通 gather; evaluator selector 没有看到完整 ancestry。每项错误都说明验证器自身也需要测试、审查和 fail-closed release, 不能把“形式化工具”当作天然正确的裁判。

当前已经对五个 headline paths 中的关键 custom operators、shuffle/unshuffle、ordinary/zigzag relations、MoE ownership 和 input contracts 做过 source-level audit, 并以这些发现建立回归; 但项目尚未发布按全部 operator/layout 统计的完整 fidelity-audit 分母、抽样协议和双人审查记录。因此本文不能声称 translator 已被全面审计。完整 coverage 与 remaining trust assumptions 列为 RQ1 和成稿前门禁。

9 命题如何生成

9.1 完整 backward ancestry

给定目标 tensor, 生成器沿 producer edge 向后闭合。若目标依赖另一 rank 的 collective input, 则两侧都必须继续回溯。遍历终点只能是 authority 中的真实 external inputs。旧 cut proof 曾把某个中间 cache 或后 12 层入口 tensor 当作 initStore 输入, 却没有约束其 lineage; 改变该中间值而保持 shape 不变就可能改变最终输出, 因此命题过弱。

9.2 输入条件与 non-vacuity

合法 caller contract 可以要求 canonical shape、initial shard relation、packed cu_seqlens well-formedness、label bounds 等执行前事实。它不能要求“L23 stream 已经满足 Zigzag2Rel”或“某个 loss gather 已相等”, 因为这些正是图执行应证明的结果。FivePublicContractsJointWitness 分别为每个

Goal 给出一对具体 Stores 并满足该 Goal 的全部 premises，避免用不可能 premise 证明任意结论。它不声称同一对 Stores 同时满足五套 shape environment，也不验证真实训练 batch 必然满足这些 conditions。

9.3 full、cut 与 false findings

full theorem 从真实 external inputs 覆盖目标完整依赖；cut theorem 从中间边界证明局部后缀；legacy/plain theorem 可能使用较弱 evaluator。它们可以作为中间结果，但公开名称必须指向 full theorem。

五个公开 full theorem 不意味着原始 obligation 全部为真。历史 corpus 把 1151 个 intermediate TIDs 与 5 个 top-level goals 组成 1156 项，其中两条旧 Goal 3/4 ordinary rank-order gather 等式被 zigzag ownership 证伪，历史口径为 1154/1156。正确处理是保留 false finding、在忠实 relation 下构造新的 ancestry-closed 命题，并清楚区分“旧命题为假”和“新命题已证”。由于当前 coverage 脚本与历史 corpus 已经漂移，1154/1156 只作为历史 finding 分母，不能作为当前 release 指标；当前 release 只以五个具名 public theorem、registry、axiom audit 与 final receipt 计量。

10 命题如何证明

10.1 局部关系定理

proof library 把 operator semantics 与输入 relation 组合。elementwise operator 通常保持同一 layout；linear 的输出 relation 取决于 weight 和 activation 如何分片；reshape 必须保持 element order 并更新 shape；shuffle 改变 relation；collective 需要跨 rank 输入；MoE 规则还要使用 routing 与 expert ownership。

10.2 图级组合

composer 按 producer order 维护 relation environment。每个 certificate node 记录 authority graph fact、选中的 local theorem、premises、side conditions 和 output relation。prefix bridge 只有在两图 prefix 可证明相等时使用；filtered bridge 只允许忽略目标 backward scope 外节点；Graph A 的事实不能因为 shape 或节点名相似就无桥搬到 Graph B。

10.3 YOCO 五个公开目标

Goals 1 和 2 从 embedding 穿过 24 层 decoder、mixed attention、unshuffle、RMSNorm 和 cross-entropy head，复用长 producer prefix 但分别证明不同 loss tail。Goal 3 观察 24 层 routing-map stack；Goal 4 观察 24 层 expert-probability stack；两者只在自己的 observable scope 内建立 graph bridge。Goal 5 使用一张较小的完整 ancestry 图，证明 hidden-sharded embedding 与双 rank AllToAll 输出关系。

Goals 1–4 使用 denoteGraphDistributedFaithful，因为目标 ancestry 含跨 rank collective、replica-buddy 或 permutation；Goal 5 使用普通 denoteGraph，但仍覆盖完整小图。公开入口位于 Instances.lean，并由 MainTheorem.lean 聚合。

10.4 防止“证明错命题”

release gate 逐项检查：

- public theorem type 字面等于 goal_N_stmt_full；
- 无 sorry、sorryAx、新增手写 axiom 或 False.elim；
- caller contracts 有可满足性见证；
- final snapshot 每个 Lean source 都作为 direct elaboration root；

- proof registry 固定 exact paths 与 source digests;
- stale .olean、orphan module 和 statement drift 都被拒绝。

11 YOCO-MoE-A0.4B 案例

11.1 authority 与图规模

最终 authority 来自双 NVIDIA RTX PRO 6000 Blackwell、双 rank NCCL production run。sealed release 对应 TrainVerify revision 63cabbff6888e3d51359dd2bf9f6d6ae75c2e98a; 此后的文档 commit 不改变该 authority 或 proof source。固定 llm-train revision 为 9a1be1d5fd1c063d80be82797692cdc7d23cfbef, nnScaler revision 为 d3d468ed23edb2f28aa8566b2dfb6ed49c5955cf。原始 SM 有 2074 nodes, PM 有 4363 nodes。SM/PM raw pickle SHA-256 分别为 333a14387e13cb265e74588f02133a0c47b02329d3d1811e6d7a736387494f64 和 a47d033c83a0cd6dff9111ba49071f3a80eb60869435fc2adeb4f725a71c3462。

11.2 证明与发布结果

最终公开五个 full theorem; all-source direct Lean gate 检查 463 个模块; proof registry 包含 455 个模块; exact ledger 有 456 项; emitter tests 为 54/54, 完整 Python suite 为 111/111。registry 本身不内嵌在 sealed snapshot 中, 而由 final receipt 以外部路径和 digest 绑定。最终 manifest digest 为 e54a53a32a316ea98e8ee60376ae477f51eebb823355bdb3632de414f1da832d。

这些数字只描述同一 final receipt 绑定的工件。历史 coverage、旧 manifest 和早期 candidate snapshot 不应与最终结果拼接。

11.3 发现、撤回与责任边界

验证过程确认了一项 llm-train CC12 grouped-GEMM selector 兼容性问题, 也确认了一个历史 nnScaler RVD/annotation/adaptor 无法表达 zigzag ownership 的 finding。后者只有双进程 CPU/Gloo 真实源码复现, 不能写成 CUDA/NCCL 复现。

调查也产生过两次需要公开的撤回。其一是把 cache K/V 错误推断为 zigzag-owned; 源码回溯证明 K/V projection 发生在 shuffle 之前。其二是把 trace scope 外的 balance-loss consumer 用于推断 scope 内 goal 为假。两项都不能包装成 llm-train 语义 bug。撤回不是“删掉失败历史”, 而是验证方法避免把自己的错误假设固化成正式结论的证据。

12 实现与信任边界

12.1 实现组件

实现包括 authority generator、nnScaler/Verdict backend、graph-to-Lean emitter、tensor denotation library、relation/proof modules、axiom audit、direct all-source builder、proof registry 和 content-addressed publisher。大型 Lean declaration 按语义边界拆分, 并限制 direct build worker 数量, 避免聚合 target 在高核机器上触发失控并发。

12.2 可信计算基线

Lean theorem 依赖常规内核项 propext、Classical.choice 和 Quot.sound。有限命题使用 native _decide 时, 还信任 Lean.ofReduceBool 与 Lean.trustCompiler。

Python generator、coverage scripts 与外部 checker 不属于 proof core；它们的输出必须被 Lean 检查。sealed provenance 保证检查和发布的字节一致，但不自动证明从 Python 到 Lean 的语义翻译忠实。

层	能说明什么	不能说明什么
GPU authority	图来自固定 revision 与真实环境	图变换数学上正确
Python emitter	可复现地产生声明和 certificate	声明忠实且证明成立
Lean kernel/compiler baseline	proof term 证明给定声明	声明对应真实 GPU 语义
registry/receipt	检查与发布字节一致	operator translation 正确

13 评估计划与当前证据

当前 YOCO authority 捕获了真实、复杂、包含 Transformer、MoE、context parallelism、自动生成 backward 与通信结构的 BackBone 编译结果；当前 Lean 声明与五个完整 observable 只覆盖其中的 forward/adapter/collective ancestry，不验证 backward 节点。它也不覆盖整个 WrapperModel、optimizer update、数据加载器或完整训练循环。但一篇 OSDI/SOSP 论文不能只报告“Lean build 通过”。最终评估应围绕以下问题组织。

RQ1: 翻译是否忠实？ 对代表性 operator、collective 与 layout 运行源码审计、小张量对照和 mutation tests，报告每类 mutation 在哪一层被拒绝。当前已有若干真实 verifier-side bug 作为回归案例，但仍需系统抽样。

RQ2: 覆盖多大规模和哪些语义？ 按 faithful denotation、value-lossy legacy 与 unsupported 分类 authority 节点和 obligations；报告 nodes、edges、operator classes、relations、proof bytes、Lean 时间和 peak RSS。五个 public theorem 与两个 false ordinary-gather findings 必须同时出现。

RQ3: 相比传统检查多发现什么？ 与 shape checking、CPU smoke、有限 numerical differential test 和 compiler self-check 比较，分析每种方法可见的 failure modes。不能把未完成的训练质量影响写进结果。

RQ4: 多少工作已经自动化？ 分别统计 generated certificates、generic theorem reuse、template-generated modules、model-specific handwritten proof 和人工 fidelity audit。单一 coverage 百分比不能代表自动化程度。

RQ5: 失败诊断是否有用？ 注入 unsupported operator、missing relation、ambiguous ownership、false goal、certificate corruption 与 provenance mismatch，检查系统是否报告第一个失败 node/tensor、expected/actual relation 与候选 rules。

RQ6: 新模型成本是多少？ 至少选择一个未用于开发规则库的 architecture 做 clean-room test，只提供 SM/PM authority 和 contracts。系统应一键生成完整 proof，或返回正确分类且可定位的失败。没有该实验前，本文只声称大规模 case study 和可复用 foundation。

RQ7: 信任链能否拒绝篡改？ 测试 stale .olean、statement drift、registry corruption、unexpected axiom、impossible contract 和 pickle/provenance mismatch。

RQ8: 开销是否可接受? 分 authority capture、goal generation、certificate generation、Lean elaboration 与 publication 报告 cold/incremental 时间、CPU、GPU、peak RSS 与工件大小。

成稿前待补: 当前缺少 unseen-architecture clean-room 实验、系统化 operator fidelity 抽样、完整阶段耗时/内存表和公平基线。上述结果未完成前, 摘要与结论不得使用“通用 proof compiler”。

14 泛用性与剩余人工劳动

变化	已可复用	应通用但尚未完成	必须单独投入
新模型, 同 op/layout	GraphDecl、store、已有语义/关系	全图 relation inference 与 certificate synthesis	目标 observable 与真实输入 contract
新 operator	图框架与组合定理	registry 和 side-condition plumbing	源码语义审计与局部定理
新 layout	authority 和 graph traversal	typed relation inference 与 adapter rules	重构语义及 runtime 对照
新训练框架	Lean core 与部分关系库	通用 IR adapter 接口	graph/provenance extractor
新 world size	relation 抽象的一部分	从 cp=2 推广到参数化 rank set	collective scope 和 permutation 证明
新硬件/backend	抽象 operator 语义	通用 authority capture	selector、profile 和 kernel fidelity

已经通用的基础包括 GraphDecl、Store、graph fold、部分 tensor semantics、若干 relations、authority/registry 和 kernel audit。设计上应当通用但当前仍特化的核心包括 typed rule registry、graph-wide ownership inference、proof-certificate DAG synthesis、failure localization、counterexample extraction 和参数化 world size。当前实现还带有 num_dp=1、num_mb=1 等范围限制, bridge emitter 也只支持已经实现的 topology families。

不可避免的人工工作包括: 审计真正的新 custom op; 定义新 layout 的重构语义; 为新训练框架编写并审查 graph adapter; 给出真实 harness input contracts; 选择有意义的 headline observables。新 operator 的工作应当跨模型复用, 但不能被“自动化”口号消掉。

15 局限与讨论

第一, TrainVerify 验证项目定义的抽象 tensor semantics, 而不是浮点 kernel 的逐位行为。若要覆盖后者, 需要从 operator semantics 到 compiler lowering 与硬件指令的 refinement proof。

第二, Python-to-Lean translation 尚未被形式化。人工 audit、小张量对照和 mutation tests 能发现错误, 但不能提供完备保证。未来可以缩小 translator、使用 proof-producing parser, 或让 authority graph 本身携带更强的 machine-checkable semantics。

第三, 当前成果以 YOCO 为主要开发和回归对象。重复 24 层可以证明规模, 却不能替代 unseen architecture 泛化实验。

第四, proof cost 和 diagnostic usability 仍需系统评估。一个只有作者能解释的 Lean error 不是可用的 compiler failure interface。理想 proof compiler 应输出结构化错误与最小反例。

第五, authority 供应链较重。真实 hardware/profile 绑定提高可审计性, 但也增加复现成本。论文应区分“数学 proof 所需信息”和“发布级 provenance hardening”, 避免让安全工程掩盖核心验证方法。

16 相关工作框架

最终版本将按问题关系比较四类工作：分布式 DNN compiler 与 parallel planning; ML graph testing 与 differential validation; translation validation 和 proof-producing compilation; tensor/distributed program 的形式语义与定理证明。当前初稿不填未经核验的优先权或“首次” claim。每项比较需要读取原论文，说明输入对象、保证强度、是否处理 rank ownership/layout、是否产生 kernel-checkable certificate，以及是否在真实训练图上评估。

成稿前待补：完成主文献检索、primary-source 核验和 bibliography 后再写 related-work 比较。

17 结论

分布式训练图的正确性不能归结为 shape 一致或程序可运行。编译器改变的是值的设备所有权、排列、复制和通信路径；这些状态必须进入规格。TrainVerify 展示了一条可行路线：从真实编译工件提取 SM/PM authority，把 operator 和 layout 翻译成值语义，沿完整 producer ancestry 构造 relation theorem，再由 Lean 检查 proof certificate。

YOCO-MoE-A0.4B 案例说明这套 foundation 能够处理 Transformer、MoE 和 context parallelism 混合的真实图，并在五个公开 observable 上得到 full theorem。它也暴露了上游兼容性问题、nnScaler 表达缺口和验证器自身错误。当前系统仍不是任意网络的一键 proof compiler。下一项决定性实验不是再增加一条 YOCO 重复层证明，而是在未用于开发的 architecture 上不改源码地生成完整 proof，或返回准确、可定位的失败。

A 论文到源码映射摘要

表中的 llm/... 与 nnscaler/... 路径分别相对于固定的 llm-train 和 nnScaler revision；其余路径相对于 TrainVerify 仓库。

论文对象	主要源码位置
llm-train compile 入口	llm/nnscaler_train.py
Trainer compile-only 路径	nnscaler/cli/trainer.py; nnscaler/cli/mixed_module.py
Trace 与 IRGraph backward	nnscaler/parallel.py; nnscaler/graph/parser/converter.py; nnscaler/graph/graph.py
AutoDist 与 SPMD search	nnscaler/policies.py; nnscaler/autodist/apis.py; nnscaler/autodist/spmd_solver.py
Adapter 与 ModuleCodeGen	nnscaler/graph/gener/gen.py; nnscaler/execplan/execplan.py; nnscaler/codegen/module/module.py
TrainVerify pickle 注入	scripts/yoco_regen/patch_mgener_dump.py
Verdict graph normalization	Verdict/nnscaler_backend/load_graph.py; Verdict/nnscaler_backend/build_graph.py
Authority generation	scripts/yoco_regen/generate_authority.sh
Graph inspection/emission	scripts/yoco_regen/emit_yoco_a04b.py
Proof registry	scripts/yoco_regen/yoco_proof_registry.json
Graph-to-Lean	Verdict/graph_to_lean.py

Five public theorem interface	<code>trainverify/denote/yoco_goals/Instances.lean</code>
Aggregate theorem	<code>trainverify/denote/yoco_goals/MainTheorem.lean</code>
Non-vacuity witness	<code>trainverify/denote/yoco_goals/FivePublicContractsJointWitness.lean</code>
Full engineering audit	<code>docs/TRAINVERIFY_YOCO_FORMALIZATION_AUDIT.md</code>
Final receipt	<code>\$HOME/yoco-final-publication/receipts/yoco-a04b-final-sealed.json</code>

B 成稿前证据门禁

- 所有节点数、obligation 数、theorem 状态、测试结果和 digest 从同一个 final receipt 机械生成。
- 完成 llm-train 到 nnScaler SM/PM codegen 的精确源码时序图。
- 完成 operator/relation coverage 与人工 audit 表。
- 完成 unseen architecture clean-room experiment。
- 完成 baseline、mutation、diagnostic quality 和 performance 评估。
- 完成相关工作 primary-source audit。
- 把最终英文稿迁移到选定年份的官方 OSDI 或 SOSP 模板，并重新做页数、匿名与 artifact 规则检查。